Федеральное государственное бюджетное образовательное учреждение высшего образования
«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Процессы и профессиональная практика программной инженерии»

на тему:
«Практическая работа 8. Проектирование и программирование пользовательского интерфейса»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверил
Преподаватель:
Алешов Е.В.

Оценка:_____
Дата: 10.11.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

Цель работы

Изучить основные принципы проектирования пользовательского интерфейса и практически реализовать учебный клиент с командными кнопками и группой кнопок (AbstractButton/ButtonGroup в Swing; QAbstractButton/QButtonGroup в PySide6).

Задание

Изучить теоретический материал по проектированию и программированию пользовательского интерфейса, ответить на контрольные вопросы и реализовать демонстрационную программу с GUI PySide6 на вымышленных данных клиента ЕМ.

Краткие теоретические сведения

Пользовательский интерфейс должен обеспечивать управление пользователем, непротиворечивость, снисходительность и адаптируемость. Usability / FURPS+.

В Swing: AbstractButton, ButtonGroup. Qt/PySide6: QAbstractButton, QButtonGroup.

Ответы на контрольные вопросы

1. В чем состоят ключевые архитектурные различия между программируемым («толстым») клиентом и клиентом-браузером («тонким» клиентом)? Какие возможности локальной обработки данных имеет каждый из них?

Ответ: Программируемый («толстый») клиент выполняет приложение локально, имеет доступ к ресурсам рабочей станции, может загружать данные с сервера, отключаться от источника, кэшировать и обрабатывать данные локально, а затем передавать результат в пользовательский интерфейс. Клиент-браузер («тонкий» клиент) зависит от сервера: он получает данные и инструкции представления (обычно HTML) и почти не выполняет собственной

обработки, кроме простых проверок ввода. Толстый клиент поддерживает богатую локальную логику; тонкий — в основном отображение и минимальную валидацию на стороне браузера.

2. Что включает в себя понятие «применимость» (usability) системы в рамках характеристик FURPS+?

Ответ: В рамках FURPS+ применимость (usability) включает простоту использования и изучения системы, обеспечение эффективности работы пользователя, скорость выполнения действий, устойчивость к ошибкам и возможность восстановления после них, а также адаптируемость среды пользовательского интерфейса. По сути, это «дружественность» системы к пользователю.

3. Каковы требования к информационной обратной связи программы в случаях длительной обработки данных или фоновых системных операций?

Ответ: При длительной обработке или фоновых системных операциях программа должна своевременно информировать пользователя о том, что происходит: показывать индикатор прогресса, статус, процент выполнения или поясняющее сообщение. Пользователь не должен оставаться без обратной связи, даже если операция временно не может быть прервана. Это снижает тревожность и сохраняет ощущение управления.

4. Каким образом реализуется принцип «Снисходительность интерфейса» при ошибочных или недопустимых действиях пользователя?

Ответ: Принцип снисходительности предполагает, что интерфейс допускает ошибки пользователя и помогает их исправлять: подтверждение опасных действий, возможность отмены (Undo), понятные сообщения об ошибках, блокировка недопустимых операций, восстановление после сбоев. Система не должна «наказывать» пользователя за ошибочный ввод, а направлять к корректному действию.

5. Какие три группы факторов определяют необходимость адаптивности пользовательского интерфейса (квалификация пользователя, география/локализация, специальные возможности)?

Ответ: Необходимость адаптивности UI определяют три группы факторов: 1) квалификация пользователя (новичок/эксперт, разные сценарии работы); 2) география и локализация (язык, форматы даты/чисел, культурные особенности); 3) специальные возможности (доступность для пользователей с ограничениями зрения, слуха, моторики и т.п.).

6. Что означает понятие «сменное впечатление и ощущение от использования» (Pluggable Look and Feel) в Java Foundation Classes (JFC)?

Ответ: Pluggable Look and Feel (PLAF) в JFC означает возможность менять внешний вид и поведение компонентов Swing во время выполнения без переписывания логики приложения. Один и тот же набор компонентов может отображаться в стиле Metal, Nimbus, системном виде платформы и др., что повышает кроссплатформенность и единообразие.

7. Для какой цели в тяжеловесных контейнерах (например, JFrame) используется метод getContentPane() при добавлении графических элементов?

Ответ: В тяжеловесных контейнерах (JFrame, JDialog и т.п.) метод getContentPane() возвращает панель содержимого, в которую следует добавлять графические компоненты. Непосредственное добавление в сам JFrame исторически некорректно: рабочая область отделена от рамки окна, строки меню и служебных областей, поэтому элементы размещают именно на content pane.

8. Какие задачи решают специализированные легковесные контейнеры JTabbedPane, JScrollPane, JSplitPane и JTable?

Ответ: JTabbedPane организует интерфейс во вкладки; JScrollPane обеспечивает прокрутку содержимого, не помещающегося в видимую область; JSplitPane разделяет область на изменяемые панели; JTable отображает и

редактирует табличные данные. Это специализированные легковесные контейнеры/компоненты для типичных задач компоновки и представления.

9. Какую функцию выполняют менеджеры расположения (Layout Managers) и какие стандартные менеджеры предоставляет библиотека Swing?

Ответ: Менеджеры расположения автоматически размещают и изменяют размеры компонентов в контейнере при изменении размера окна. В Swing стандартно доступны, в частности, BorderLayout, FlowLayout, GridLayout, GridBagLayout, BoxLayout, CardLayout, а также null-layout (ручное позиционирование). Выбор менеджера зависит от структуры контейнера.

10. Каковы особенности менеджеров компоновки BorderLayout, FlowLayout и GridLayout, и как отменяется автоматическое размещение компонентов (null-расположение)?

Ответ: BorderLayout делит контейнер на север/юг/запад/восток/центр; FlowLayout размещает компоненты подряд с переносом строк; GridLayout раскладывает элементы в равномерную сетку строк и столбцов. Автоматическое размещение отменяется установкой null-layout (setLayout(null)), после чего позиции и размеры задаются вручную через setBounds().

11. В чем состоят функциональные различия между пунктами меню JCheckBoxMenuItem и JRadioButtonMenuItem?

Ответ: JCheckBoxMenuItem — пункт меню с независимой отметкой (можно включать/выключать несколько пунктов). JRadioButtonMenuItem — пункт взаимоисключающего выбора: в группе обычно активен только один. Таким образом, check-пункты дают множественный независимый выбор, radio-пункты — единственный вариант из набора.

12. От какого базового класса наследуются все командные кнопки и какую функцию выполняет класс ButtonGroup?

Ответ: Все командные кнопки в Swing наследуются от абстрактного класса AbstractButton. Класс ButtonGroup объединяет набор кнопок (обычно JRadioButton) в группу взаимоисключающего выбора: в каждый момент времени может быть выбрана только одна кнопка группы. В Qt/PySide6 аналоги — QAbstractButton и QButtonGroup.

Ход работы

По материалам работы изучены принципы проектирования UI и компоненты Swing. Реализована программа на Python/PySide6 с меню, панелью инструментов, деревом навигации, вкладками и QButtonGroup.

Результат выполнения

Рабочая программа в app/. Запуск: python main.py. Зависимость: PySide6.

Скриншоты работы программы

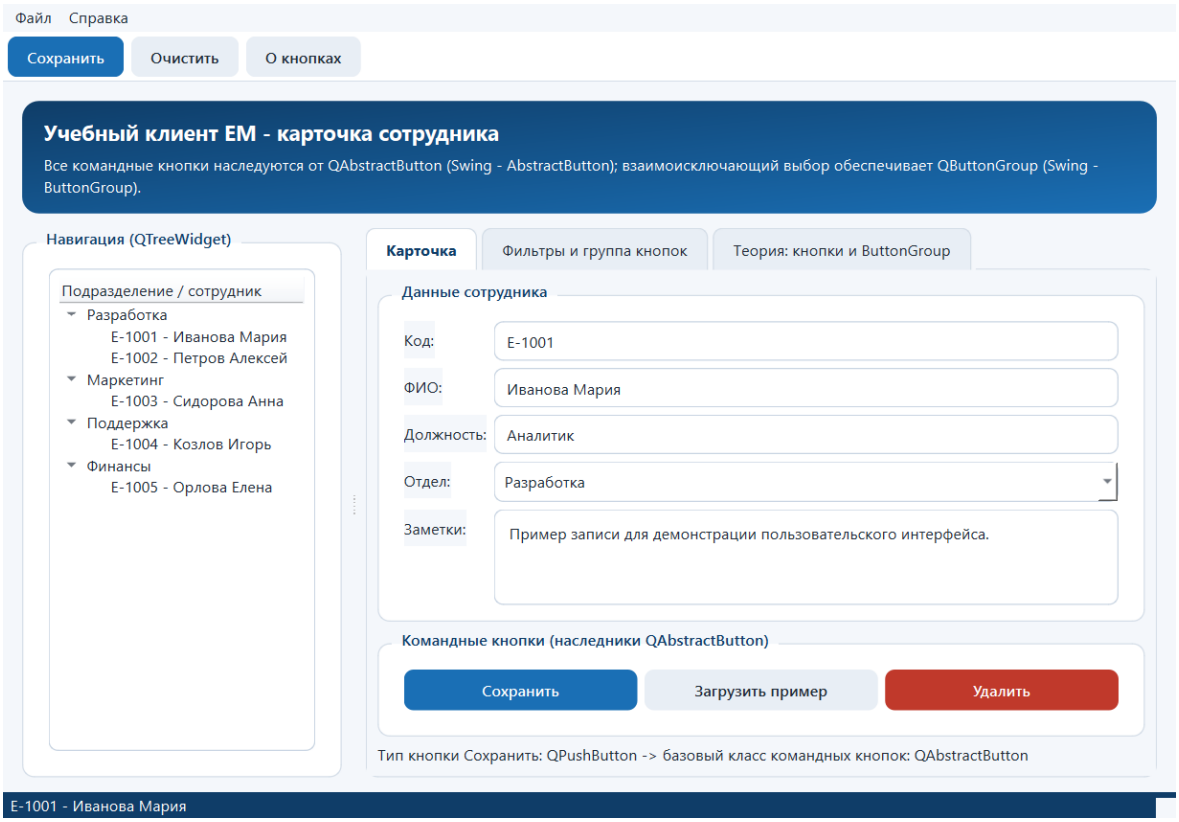


Рисунок 1. Главное окно

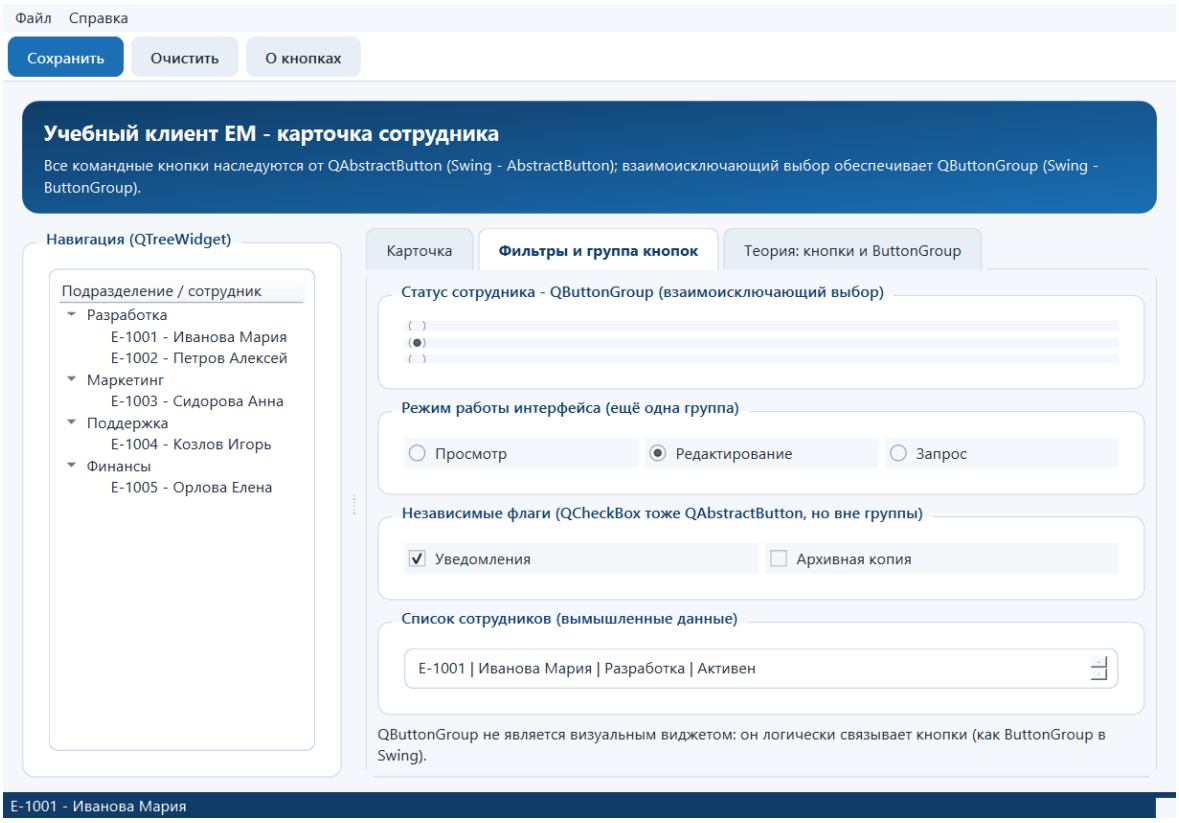


Рисунок 2. Фильтры и QButtonGroup

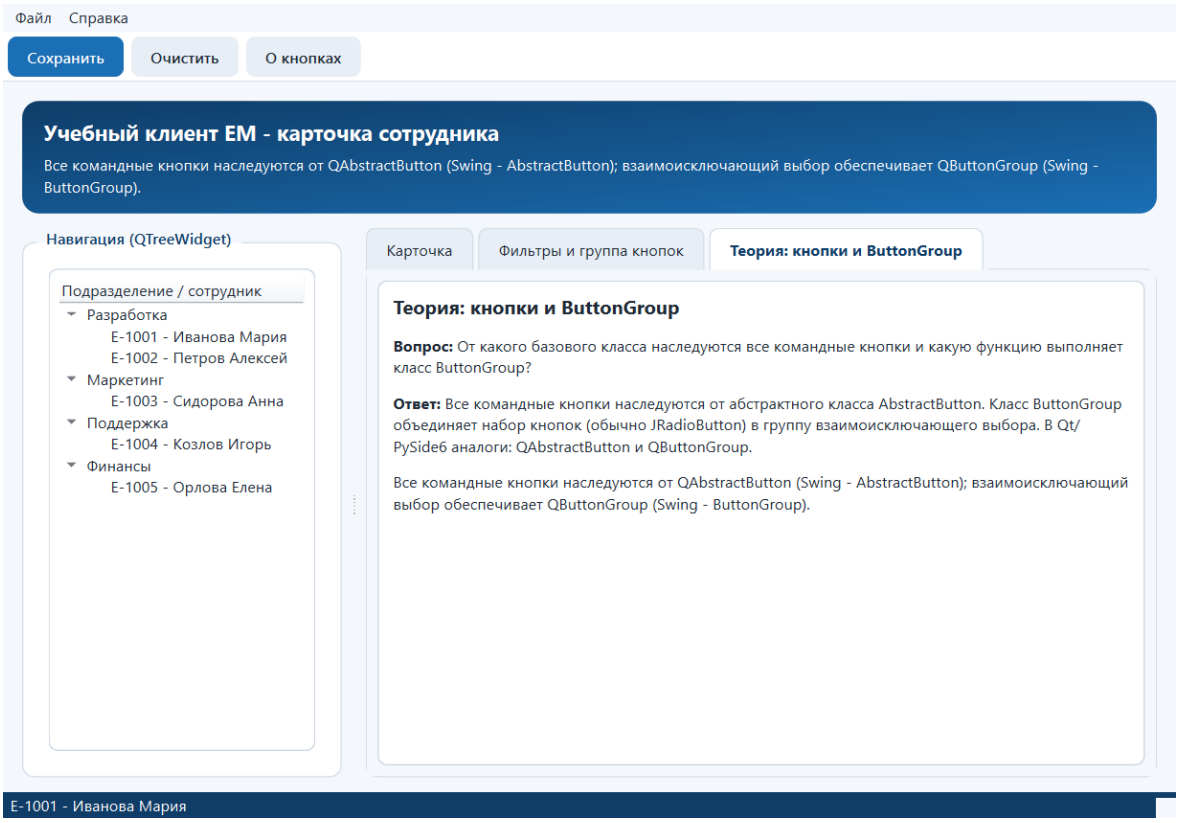


Рисунок 3. Теория по кнопкам

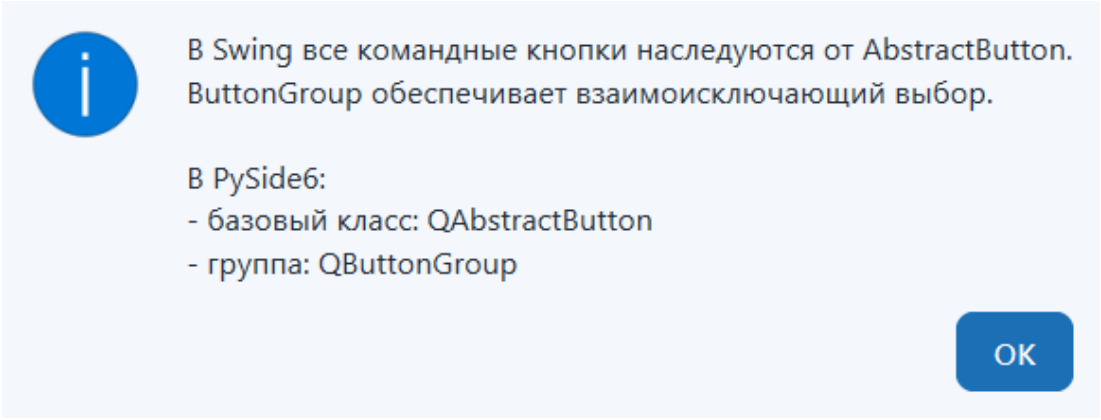


Рисунок 4. Диалог о кнопках

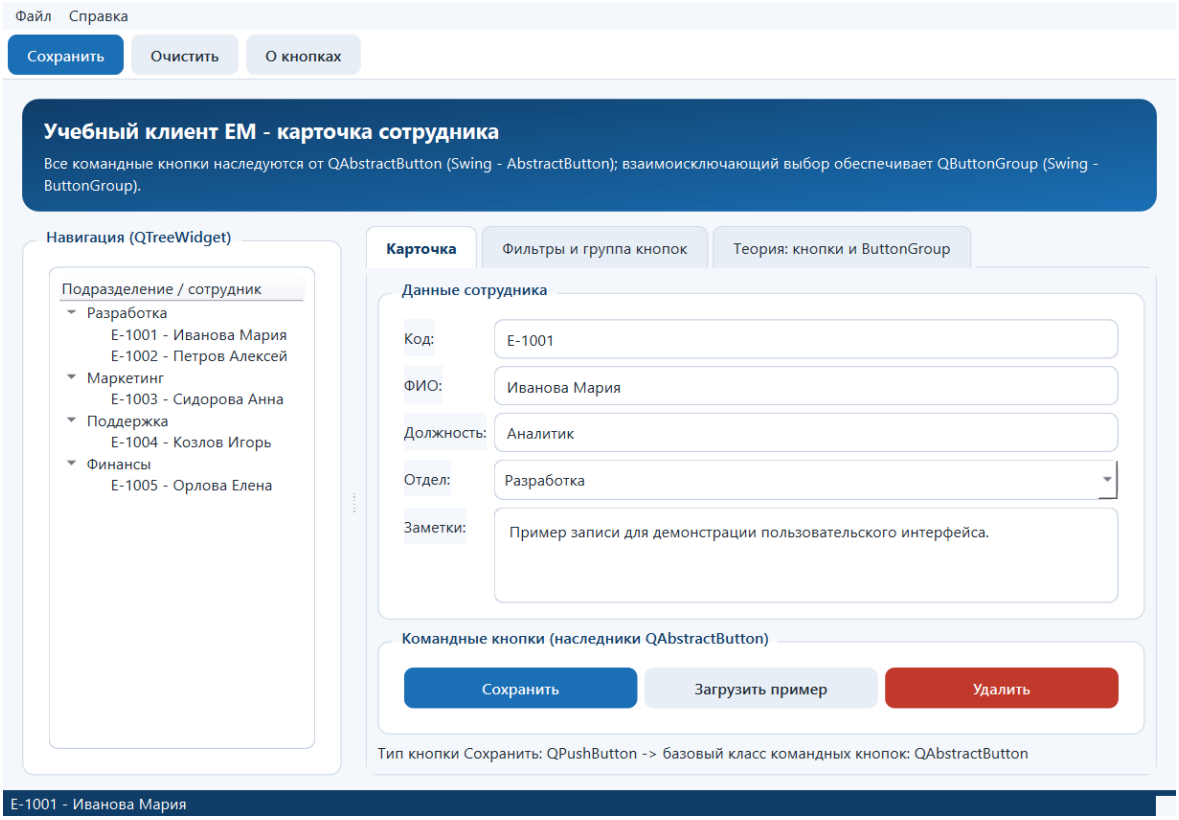


Рисунок 5. Общий вид

Вывод

В ходе работы изучены принципы проектирования UI и получены ответы на контрольные вопросы. Реализована программа на PySide6, демонстрирующая QAbstractButton и QButtonGroup.